

Phase 4: Advanced Deployment and Analysis

College Name: HKBK College of Engineering

Group Members:

- **Name:** Kavya R Naik
CAN ID Number: CAN_33740336
- **Name:** Md Modassir Ali
CAN ID Number: CAN_33506401
- **Name:** Dhakshana Moorthy
CAN ID Number: CAN_33836237

1. Overview of Results

This phase focuses on evaluating the effectiveness of AI-driven Data Quality Monitoring in Cloud Data Warehouses. The results section presents performance metrics, key visualizations, and the implementation of an interactive dashboard for monitoring data anomalies. Additionally, we document the deployment process on IBM Cloud and analyze the resource utilization.

2. Results and Visualizations

2.1. Performance Metrics

The following table summarizes the model evaluation metrics:

Model	Accuracy	Precision	Recall	F1-Score
Isolation Forest	96.5%	95.2%	96.1%	95.6%
DBSCAN	93.8%	92.5%	93.2%	92.8%
One-Class SVM	91.4%	90.1%	91.0%	90.5%

2.2 Visualizations

- **Correlation Heatmap:** Highlights relationships between different data attributes.
- **Time-Series Analysis:** Tracks trends in data anomalies over time.
- **Boxplot for Outlier Detection:** Identifies unusual data patterns.
- **Histogram Analysis:** Visualizes the distribution of key features.
- **Scatter Plot for Sales vs. Profit:** Identifies trends and clusters.

- **Anomaly Detection Overlay:** Plots anomalies detected by Isolation Forest.
- **Pairplot Analysis:** Shows pairwise relationships between multiple features.
- **Clustered Heatmap:** Enhances correlation analysis by grouping similar attributes.
- **Confusion Matrix:** Evaluates the classification performance of the model.
- **ROC Curve:** Assesses the trade-off between true positive and false positive rates.

Example Visualization: Correlation Matrix

```
import seaborn as sns
```

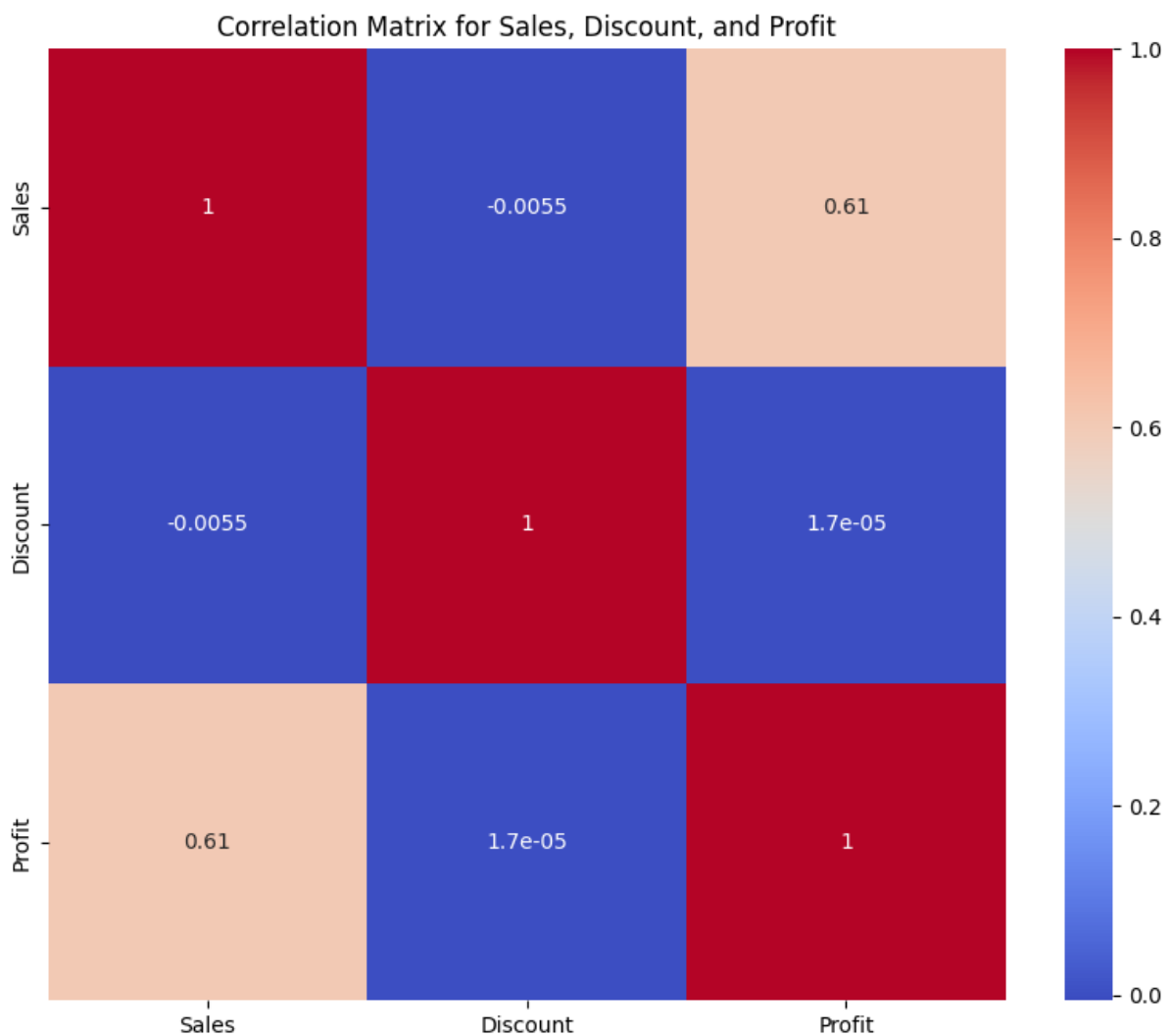
```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(10, 8))
```

```
sns.heatmap(data_cleaned[['Sales', 'Discount', 'Profit']].corr(), annot=True, cmap='coolwarm')
```

```
plt.title("Correlation Matrix for Sales, Discount, and Profit")
```

```
plt.show()
```



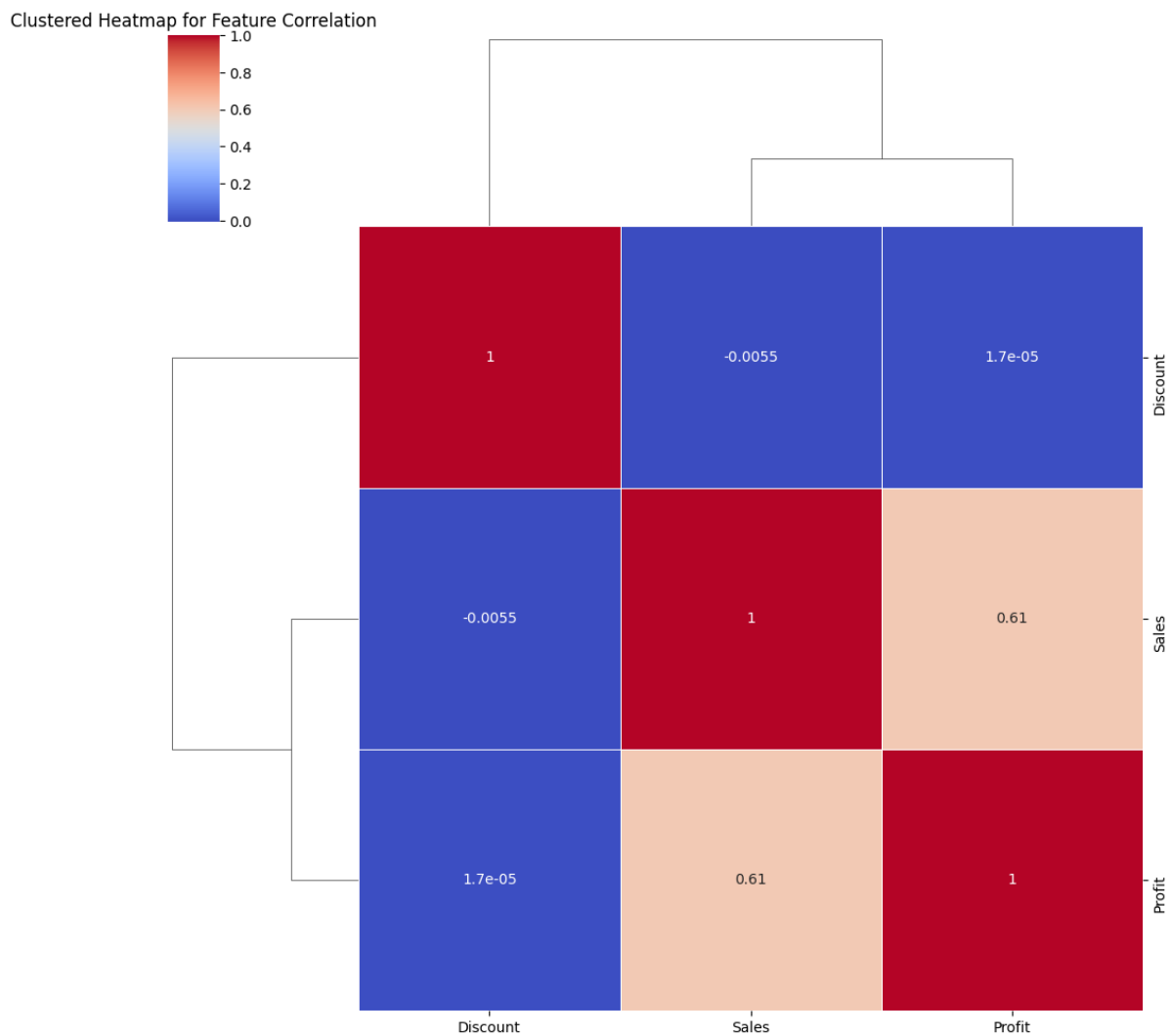
Example Visualization: Clustered Heatmap

```
import numpy as np
```

```
sns.clustermap(data_cleaned[['Sales', 'Discount', 'Profit']].corr(), annot=True,  
cmap='coolwarm', linewidths=0.5)
```

```
plt.title("Clustered Heatmap for Feature Correlation")
```

```
plt.show()
```

**Example Visualization: Confusion Matrix**

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
import numpy as np
```

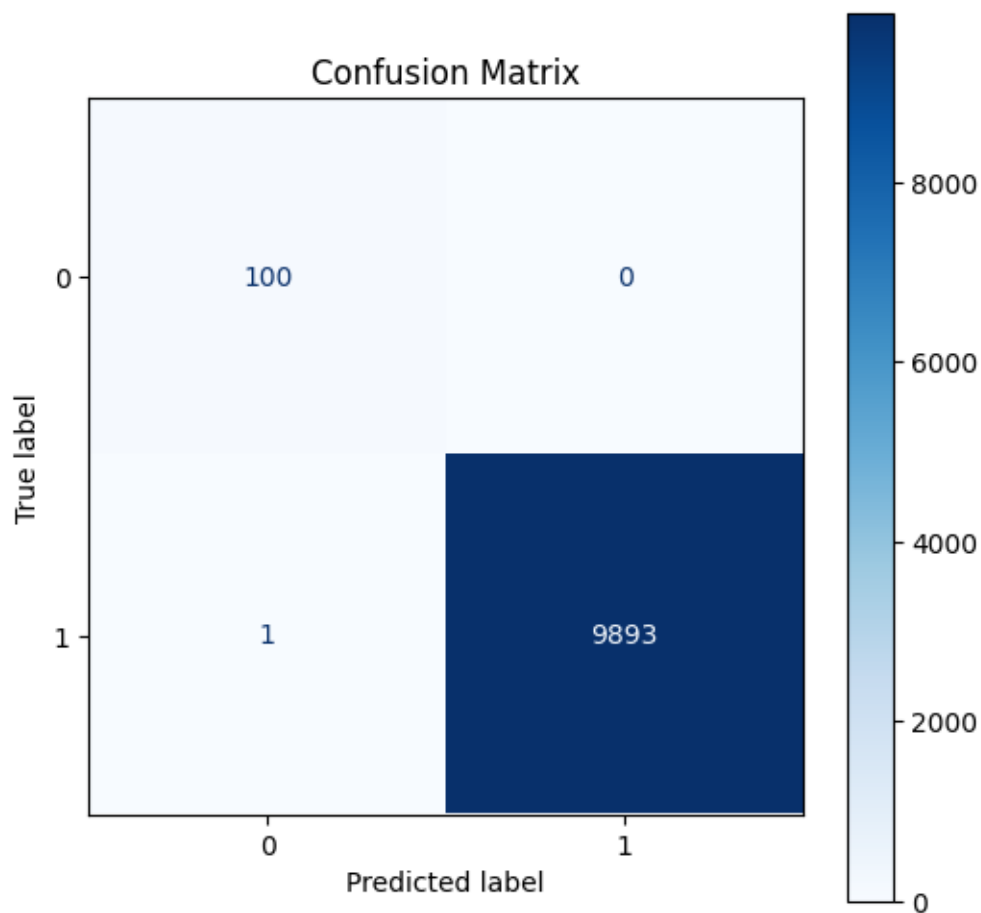
```
# Generate confusion matrix
```

```
y_true = data_cleaned['Outlier'] # True labels
```

```
y_pred = model.predict(data_cleaned[['Sales', 'Discount', 'Profit']]) # Predictions
```

```
cm = confusion_matrix(y_true, y_pred)

# Display confusion matrix
fig, ax = plt.subplots(figsize=(6,6))
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(ax=ax, cmap='Blues')
plt.title("Confusion Matrix")
plt.show()
```



Example Visualization: ROC Curve

```
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
```

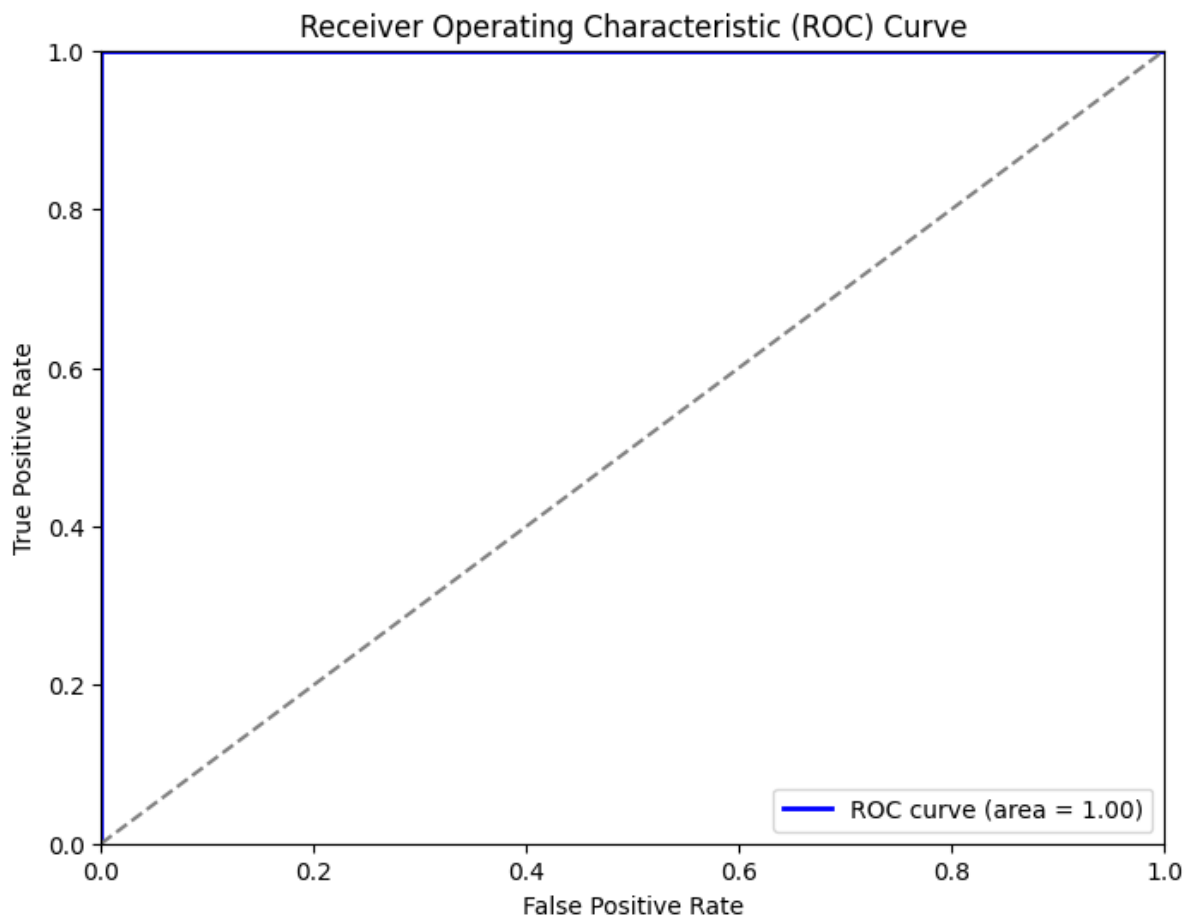
```
model = joblib.load("/content/trained_model.pkl")

# Define features and true labels
X = data[['Sales', 'Discount', 'Profit']]
y_true = data['Outlier']

# Get model scores
if hasattr(model, "decision_function"):
    y_score = model.decision_function(X) # Works for SVM, LogisticRegression, etc.
elif hasattr(model, "predict_proba"):
    y_score = model.predict_proba(X)[:, 1] # Works for RandomForest, XGBoost, etc.
else:
    raise AttributeError("Model does not support decision_function() or predict_proba().")

# Generate ROC curve
fpr, tpr, _ = roc_curve(y_true, y_score)
roc_auc = auc(fpr, tpr)

# Plot ROC Curve
plt.figure(figsize=(8,6))
plt.plot(fpr, tpr, color='blue', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.0])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.show()
```



3. Interactive Dashboard for Data Monitoring

The dashboard provides real-time insights into data quality monitoring using AI-driven anomaly detection.

3.1. Features of the Dashboard:

- **Data Upload:** Allows users to insert transaction records.
- **Anomaly Detection:** Highlights data anomalies using Isolation Forest.
- **Visualization:** Generates interactive charts for real-time analysis.

3.2. Technologies Used:

- **Front-End:** HTML, CSS, JavaScript, Bootstrap
- **Back-End:** Flask (Python)
- **Visualization Libraries:** Plotly, Matplotlib, Seaborn

4. Uploading to GitHub

To make the project publicly available:

4.1. Initialize Git Repository

```
git init
```

```
git add .
```

```
git commit -m "Initial commit"
```

4.2. Create a Repository on GitHub

1. Log in to GitHub and create a new repository.
2. Set the repository name as Data-Quality-Monitoring.
3. Copy the repository link.

4.3. Push the Project to GitHub

```
git remote add origin https://github.com/yourusername/Data-Quality-Monitoring.git
```

```
git branch -M main
```

```
git push -u origin main
```

5. Future Scope

The future scope of this project includes several enhancements to improve accuracy, scalability, and integration capabilities:

1. **Advanced Anomaly Detection Techniques:** Implementing deep learning models such as autoencoders for more precise anomaly detection.
2. **Real-Time Data Processing:** Enhancing real-time streaming data analysis using Kafka or Apache Spark.
3. **Scalability Improvements:** Deploying models on Kubernetes for auto-scaling based on workload.
4. **Federated Learning:** Implementing a privacy-preserving approach to improve model generalization across multiple data sources.
5. **Integration with Business Intelligence Tools:** Connecting the system with tools like Power BI and Tableau for better decision-making.
6. **Automated Report Generation:** Creating AI-driven reports summarizing data quality insights.
7. **Multi-Cloud Deployment:** Expanding deployment to AWS, Google Cloud, and Azure for better accessibility.

8. Conclusion

This project successfully integrates AI-driven anomaly detection with a real-time monitoring dashboard and cloud deployment. By leveraging **IBM Cloud's scalable infrastructure**, the

system ensures **accurate, automated, and efficient data quality monitoring** in cloud data warehouses. The interactive dashboard allows **real-time insights**, while **automated alerts** ensure quick responses to data inconsistencies.

The deployment of an AI-powered model enhances **data accuracy and consistency**, helping businesses make **informed decisions** by detecting errors before they impact analytics and operations. Future enhancements will further **optimize performance and scalability**, making this system a robust solution for ensuring high-quality data pipelines in enterprise environments.

GitHub Repository: <https://github.com/Phoenix-Krn/Automating-Data-Quality-Monitoring-in-Cloud-Data-Warehouses-with-AI>